

树

2024年11月6日 19:19

mak a
bak a

□ 树:

中序前继:

如果节点有左子树, 那么它的中序前驱就是其左子树中最右下的节点。

如果节点没有左子树, 那你就沿着树向上找, 直到找到一个节点, 它是作为右孩子出现的。
这时, 这个节点的父节点就是我们要找的中序前驱。

大顶堆的删除:

```
template <class T>
void MaxHeap<T>::Pop()
{
    // Delete max element.
    if (IsEmpty()) throw "Heap is empty. Cannot delete.";
    // 检查堆是否为空, 如果是, 抛出异常提示无法删除最大元素。

    heap[1].~T();
    // 调用析构函数, 删除堆顶元素。

    // remove last element from heap
    T lastE = heap[heapSize--];
    // 移除堆中的最后一个元素, 并将其保存到 `lastE` 变量中。同时, 堆的大小减1。

    // trickle down
    int currentNode = 1; // root
    int child = 2; // a child of currentNode
    // 初始化 `currentNode` 为根节点, `child` 为根节点的左孩子。

    while (child <= heapSize)
    {
        // set child to larger child of currentNode
        if (child < heapSize && heap[child] < heap[child + 1]) child++;
        // 找到 `currentNode` 的左右孩子中较大的那个。

        // can we put lastE in currentNode?
        if (lastE >= heap[child]) break; // yes
        // 如果 `lastE` 大于等于 `child` 节点的值, 跳出循环。

        // no
        heap[currentNode] = heap[child]; // move child up
        // 将较大的孩子节点移动到当前节点位置。

        currentNode = child; child *= 2; // move down a level
        // 更新 `currentNode` 为 `child`, 并将 `child` 更新为其左孩子, 继续向下处理。
    }

    heap[currentNode] = lastE;
    // 将 `lastE` 放置在最终确定的位置上。
}
```

大顶堆

2024年11月6日 19:21

见代码: MaxHeap

```
#include<iostream>

using namespace std;

// 模板类 最大堆
template<class T>
class MaxHeap
{
private:
    T* heap;      // 存储堆元素的数组
    int capacity; // 堆的最大容量
    int heapSize; // 当前堆中的元素个数
public:
    // 构造函数, 默认容量为10
    MaxHeap(int cap = 10)
    {
        if (cap < 1)
        {
            cerr << "Capacity must be >= 1."; // 如果容量小于1, 输出错误信息并退出
            exit(0);
        }

        capacity = cap;
        heapSize = 0;
        heap = new T[capacity + 1]; // 创建一个容量为cap+1的数组, heap[0]不使用
    }

    // 析构函数, 释放内存
    ~MaxHeap()
    {
        delete[] heap;
    }

    // 检查堆是否为空
    bool isEmpty()
    {
        if (heapSize == 0)
```

```

        return true;
    return false;
}

// 返回堆顶元素
T& Top()
{
    return heap[1];
}

// 扩展堆的大小
void resize()
{
    capacity *= 2; // 将容量加倍
    T* newHeap = new T[capacity + 1]; // 创建一个新的更大的数组
    for (int i = 1; i <= heapSize; i++)
    {
        newHeap[i] = heap[i]; // 复制旧数组的元素到新数组
    }
    delete[] heap; // 释放旧数组的内存
    heap = newHeap; // 将指针指向新数组
}

// 插入一个新元素到堆中
void push(const T& e)
{
    if (capacity == heapSize)
    {
        resize(); // 如果堆已满，先扩展堆的大小
    }

    int currentNode = ++heapSize; // 增加堆的大小，并找到最大堆的最后一个位置
    // 在向上调整过程中，找到新元素的正确位置
    // 在让堆大小增加的同时，找到最大堆的最后一个位置，所以是前置自增
    while (currentNode != 1 && heap[currentNode / 2] < e)
    {
        heap[currentNode] = heap[currentNode / 2]; // 如果父节点小于新元素，将父节点下移
        currentNode /= 2; // 向上移动到父节点
    }
    heap[currentNode] = e; // 将新元素插入到正确的位置
}

// 删除堆顶元素
void pop()
{
    if (isEmpty())
    {
        cerr << "heap is empty" << endl; // 如果堆为空，输出错误信息并退出
        exit(0);
    }

    T lastE = heap[heapSize]; // 获取最后一个元素
    heapSize--; // 减小堆的大小

    int currentNode = 1; // 从根节点开始

```

```

int child = 2; // 左孩子
while (child <= heapSize)
{
    // 找到更大的孩子
    if (child < heapSize && heap[child] < heap[child + 1])
        child++;
    // 如果最后一个元素大于或等于更大的孩子，停止
    if (lastE >= heap[child])
        break;
    // 将更大的孩子上移
    heap[currentNode] = heap[child];
    currentNode = child;
    child *= 2;
}
heap[currentNode] = lastE; // 将最后一个元素放到正确的位置
}

```

// 书中的方法：保存最后一个元素（lastE），然后在调整过程中不使用交换，而是将子节点移动到父节点，最后将 lastE 放置到适当的位置。

// 我的方法：直接将最后一个元素移动到根节点（heap[1] = heap[heapSize]），然后通过交换操作逐步恢复堆的性质。

// 书中的方法：将元素下移时，不断赋值子节点到当前节点，最后将 lastE 插入。
// 我的方法：通过交换操作不断调整堆，每次交换都需要使用一个临时变量 T temp。

```

/*
void pop()
{
    if (isEmpty())
    {
        cerr << "heap is empty" << endl;
        exit(0);
    }

    heap[1] = heap[heapSize];
    heapSize--;

    int currentNode = 1;
    int child = 2;
    while (child <= heapSize)
    {
        if (child < heapSize && heap[child] < heap[child + 1])
            child++;
        if (heap[currentNode] >= heap[child])
            break;
        else
        {
            T temp = heap[currentNode];
            heap[currentNode] = heap[child];
            heap[child] = temp;
            currentNode = child;
            child *= 2;
        }
    }
}*/

```

```

        // 打印堆的元素
void print()
{
    if (isEmpty())
    {
        cerr << "heap is empty" << endl; // 如果堆为空，输出错误信息并退出
        exit(0);
    }

    for (int i = 1; i <= heapSize; ++i)
    {
        std::cout << heap[i] << " "; // 打印每个元素
    }
    std::cout << std::endl;
}
};

int main()
{
    MaxHeap<int> heap(10); // 创建一个最大容量为10的堆

    heap.push(21);
    heap.push(15);
    heap.push(20);
    heap.push(14);
    heap.push(10);
    heap.push(2);

    heap.print(); // 打印堆
    cout << heap.Top() << endl; // 打印堆顶元素

    heap.pop(); // 删除堆顶元素
    cout << heap.Top() << endl;
    heap.print();
    heap.pop();
    cout << heap.Top() << endl;
    heap.print();
    heap.pop();
    cout << heap.Top() << endl;
    heap.print();
    heap.pop();
    cout << heap.Top() << endl;
    heap.print();
    heap.pop();
    cout << heap.Top() << endl;
    heap.print();
}

```

二叉搜索树BST

2024年12月16日 23:01

特点：所有左子树都小于根，所有右子树都大于根

查找：从根节点开始依次比较，大于则往右子树找，小于则往左子树找
O(logn)

插入：类似查找

构建：类似插入

删除：

叶节点：直接删除

只有左/右子树的节点：删除后让它的左/右子树代替

左右子树都有的节点a: 找到a的左子树中最大的节点x，代替原节点，删除x

或者找到a的右子树中最小的节点y，代替a，删除y

(比它大的里面的最小的)

bool _Erase(Node*& root,K key)和bool _Erase(Node*root,K key)有什么区别

Node*& root

解释

在上述示例中：

- _EraseReference(Node*& root, int key)：由于root是一个引用，当我们在函数内部修改root时，调用者的root也会相应地更新。这在删除操作中非常重要，因为我们可能需要更新树的根节点。

Node* root

解释

1. 查找要删除的节点：通过比较key和root->key来决定递归方向，如果key小于root->key，则递归到左子树，否则递归到右子树。
2. 删除节点：
 - 如果找到的节点没有左子树或右子树，则可以直接将当前节点用其子节点覆盖，并删除原始节点。
 - 如果找到的节点有左右子树，则找到右子树中的最小节点，用该节点的值替换当前节点的值，然后在右子树中递归删除这个最小节点。

通过这种方式，虽然不能直接修改传入的 root，但可以通过修改其指向的内容来间接影响树的结构。这样就实现了节点的删除操作。

来自 <<https://copilot.microsoft.com/chats/lyphq2CwzQGedkQ1whaep>>

```
bool _Erase(Node*& root,K key)
{
    if(root==nullptr)
    {
        return false;
    }
    if(key<root->_key)
    {
        return _Erase(root->_left,key);
    }
    else if(key>root->_key)
    {
        return _Erase(root->_right,key);
    }
    else
    {
        Node* del=root;
        if(root->_left==nullptr)
        {
            root=root->_right;
        }
        else if(root->right==nullptr)
        {
            root=root->_left;
        }
        else
        {
            Node* leftMax=root->_left;
            while(leftMax->_right)
            {
                leftMax=leftMax->_right;
            }
            swap(leftMax->_key,root->_key);
            return _Erase(root->_left,key);
        }
        delete del;
        return true;
    }
}
```

递归实现

1.判空，空就返回false

2.找key,小于就往左递归，大于就往右递归。

3.删除也是分三种情况，没孩子，有一个孩子，有两个孩子

4.没孩子可以与与有一个孩子的情况合并处理，这里与非递归不同的是，不需要当前节点是父节点的左还是右，因为当前节点实参的引用，不是单纯地比指针变量，可以直接赋值。

5.两个孩子的情况也省略了父节点的环节，不同的是，这次节点的值不是赋值，而是交换，交换后要删除的节点就是左子树的最右节点，或者是右子树的最左节点，这两个节点都是只有一个孩子或者是没有孩子，因此可以将问题转化为前两种情况进行处理，找的是左子树就往左递归一步，右子树就往右递归一步即可。
[【c++> 二叉搜索树】_搜索c++-CSDN博客](#)

是的，使用引用参数 `TreeNode*& node` 替代返回值的方式是完全可行的，并且在某些情况下可能更简洁。通过引用方式，递归操作直接对父节点的指针进行修改，而不需要通过返回值来更新。

以下是改进后的代码：

```
void remove(int k)
{
    removeRec(root, k);
}
void removeRec(TreeNode*& node, int k)
{
}
```

```

    if (node == nullptr)
        return;
// Step 1: Find the node to delete
    if (k < node->value)
    {
        removeRec(node->left, k);
    }
    else if (k > node->value)
    {
        removeRec(node->right, k);
    }
    else
    {
        // Step 2: Handle deletion cases
        // Case 1: Node is a leaf or has one child
        if (node->left == nullptr)
        {
            TreeNode* temp = node;
            node = node->right;
            delete temp;
        }
        else if (node->right == nullptr)
        {
            TreeNode* temp = node;
            node = node->left;
            delete temp;
        }
        // Case 2: Node has two children
        else
        {
            TreeNode* minNode = findMin(node->right);
            node->value = minNode->value; // Replace current node's value with in-
            order successor's value
            removeRec(node->right, minNode->value); // Recursively delete in-order
            successor
        }
    }
}
TreeNode* findMin(TreeNode* node)
{
    while (node->left != nullptr)
        node = node->left;
    return node;
}

```

主要变化和优点：

1. **引用参数：**
 - `TreeNode*& node` 使得递归函数可以直接修改父节点指针的值。这避免了使用返回值来更新父指针的额外操作。
 - 修改了引用指向的内存后，父指针直接反映了变化，无需在每次递归结束时显式地用返回值更新节点。
2. **实现简单：**
 - `removeRec` 不需要返回 `TreeNode*`，逻辑上更加直观。
 - 直接操作引用，避免返回值引入的潜在错误。
3. **一致性：**
 - 每次递归中，传递的 `node` 始终表示当前处理的节点，递归调用后能直接反映删除的结果。

工作原理：

- 如果找到目标节点：
 - 当该节点是叶节点或仅有一个子节点时，通过修改 `node` 引用，直接更新它的父节点指针。
 - 如果该节点有两个子节点，则找到右子树中的最小节点，替换当前节点的值，并递归删除该最小节点。
- 整个过程中，`node` 始终引用当前子树的根节点，递归完成后能直接反映删除结果。

这种方法的效率和递归逻辑与之前的代码一致，但减少了通过返回值更新父指针的复杂性，推荐在需要简化递归逻辑时使用这种方式。

来自 <<https://chatgpt.com/c/6764fbf5-a8e8-8004-8bfe-aae447bc12a5>>

遍历

2024年12月17日 19:50

中序遍历

(Inorder Traversal) 是一种深度优先遍历二叉树的方式。其具体步骤是按照以下顺序访问树中的每个节点：

1. **左子树**：首先，递归遍历左子树。
2. **根节点**：然后，访问根节点。
3. **右子树**：最后，递归遍历右子树。

```
// 前序遍历迭代器
template <typename T>
class PreorderIterator {
private:
    stack<TreeNode<T>*> s;

public:
    PreorderIterator(TreeNode<T>* root)
    {
        if (root != NULL) s.push(root);
    }

    bool hasNext() {
        return !s.empty();
    }

    TreeNode<T>* next() {
        TreeNode<T>* node = s.top();
        stack.pop();
        if (node->right != NULL) s.push(node->right);
        if (node->left != NULL) s.push(node->left);
        return node;
    }
};
```

```
// 层次遍历迭代器
template <typename T>
class LevelOrderIterator
{
private:
    queue<TreeNode<T>*> queue;

public:
    LevelOrderIterator(TreeNode<T>* root)
    {
        if (root != NULL) queue.push(root);
    }

    bool hasNext()
    {
        return !queue.empty();
    }
};
```



```

TreeNode<T>* next()
{
    TreeNode<T>* node = queue.front();
    queue.pop();
    if (node->left != NULL) queue.push(node->left);
    if (node->right != NULL) queue.push(node->right);
    return node;
}

};

// 后序遍历迭代器
template <typename T>
class PostorderIterator
{
private:
    stack<TreeNode<T>*> stack1, stack2;

public:
    PostorderIterator(TreeNode<T>* root)
    {
        if (root != NULL) stack1.push(root);
        while (!stack1.empty()) {
            TreeNode<T>* node = stack1.top();
            stack1.pop();
            stack2.push(node);
            if (node->left != NULL) stack1.push(node->left);
            if (node->right != NULL) stack1.push(node->right);
        }
    }

    bool hasNext()
    {
        return !stack2.empty();
    }

    TreeNode<T>* next()
    {
        TreeNode<T>* node = stack2.top();
        stack2.pop();
        return node;
    }
};

```

使用两个栈是为了模拟递归后序遍历的过程。

1. **第一个栈** stack1: 用于暂存节点，按照根、右、左的顺序存入节点。
2. **第二个栈** stack2: 用于调整节点的顺序，使其最终成为后序遍历的顺序，即左、右、根。

具体过程如下：

- 首先，将根节点压入 stack1。
- 在 while 循环中，将 stack1 弹出的节点压入 stack2。
- 如果该节点有左子节点或右子节点，将其压入 stack1。
- 最终，stack2 中的节点顺序就是后序遍历的顺序。

这样，通过两个栈的配合，就能够实现非递归的后序遍历。简而言之，stack1 用于辅助遍历，而 stack2 用于存储后序遍历的最终结果。

线索二叉树

2024年12月29日 19:38

```
template <class T>
```

```
T* ThreadedInorderIterator::Next()
```

```
{
```

```
    // 返回当前节点在中序遍历中的后继节点
```

```
    ThreadedNode <T>* temp = currentNode->rightChild; // 获取当前节点的右子节点
```

```
    if (!currentNode->rightThread) { // 如果当前节点的右子节点不是线索
```

```
        while (!temp->leftThread) temp = temp->leftChild; // 一直向左找到最左侧的节点
```

```
    }
```

```
    currentNode = temp; // 更新当前节点为后继节点
```

```
    if (currentNode == root) return 0; // 如果当前节点是根节点，返回0
```

```
    else return &currentNode->data; // 否则，返回当前节点的数据
```

```
}
```

```
template <class T>
```

```
void ThreadedTree <T>::InsertRight(ThreadedNode <T>* s, ThreadedNode <T>* r)
```

```
{
```

```
    // 将节点 r 插入为节点 s 的右子节点
```

```
    r->rightChild = s->rightChild; // 设置 r 的右子节点为 s 的右子节点
```

```
    r->rightThread = s->rightThread; // 设置 r 的右线索为 s 的右线索
```

```
    r->leftChild = s; // 设置 r 的左子节点为 s
```

```
    r->leftThread = true; // 将 r 的左线索设置为 true (左子节点为线索)
```

```
    s->rightChild = r; // 设置 s 的右子节点为 r
```

```
    s->rightThread = false; // 将 s 的右线索设置为 false (右子节点不为线索)
```

```
    if (!r->rightThread) // 如果 r 的右子节点不为线索
```

```
    {
```

```
        ThreadedNode <T>* temp = InorderSucc(r); // 获取 r 的中序后继节点
```

```
        temp->leftChild = r; // 将后继节点的左子节点设置为 r
```

```
    }
```

```
}
```

```

template <class T>
void ThreadedTree <T>::InsertLeft(ThreadedNode <T>* s, ThreadedNode <T>* l)
{
    // 将节点 l 插入为节点 s 的左子节点
    l->leftChild = s->leftChild; // 设置 l 的左子节点为 s 的左子节点
    l->leftThread = s->leftThread; // 设置 l 的左线索为 s 的左线索
    l->rightChild = s; // 设置 l 的右子节点为 s
    l->rightThread = true; // 将 l 的右线索设置为 true (右子节点为线索)
    s->leftChild = l; // 设置 s 的左子节点为 l
    s->leftThread = false; // 将 s 的左线索设置为 false (左子节点不为线索)
    if (!l->leftThread) // 如果 l 的左子节点不为线索
    {
        ThreadedNode <T>* temp = InorderPre(l); // 获取 l 的中序前驱节点
        temp->rightChild = l; // 将前驱节点的右子节点设置为 l
    }
}

```

```

template <class T>
ThreadedNode<T>* ThreadedTree<T>::InorderPre(ThreadedNode<T>* node)
{
    // 返回节点 node 的中序前驱节点
    ThreadedNode<T>* temp = node->leftChild; // 获取 node 的左子节点
    while (!temp->rightThread) // 如果左子节点不是线索，向右找到最右侧的节点
        temp = temp->rightChild;
    return temp; // 返回中序前驱节点
}

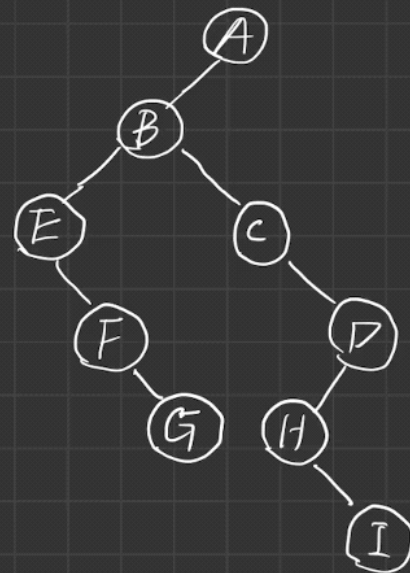
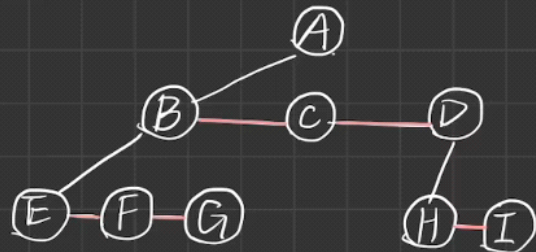
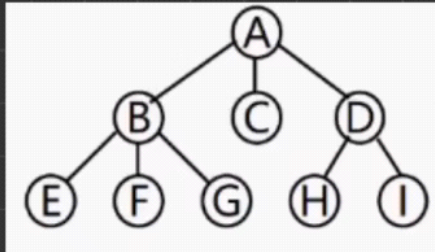
```

二叉树，森林

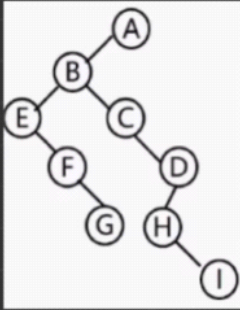
2025年1月1日 22:41

左孩子，右兄弟

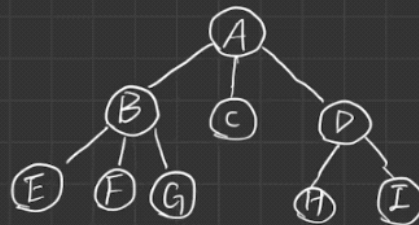
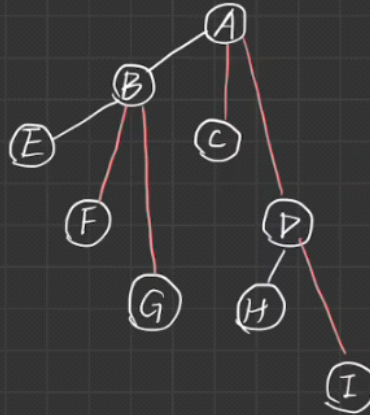
1. 树 $\Rightarrow$ 二叉树 兄弟相连留长子，转45°



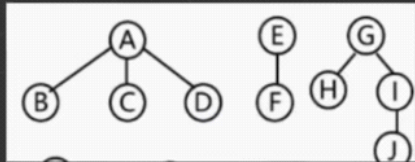
2. 二叉树 $\Rightarrow$ 树



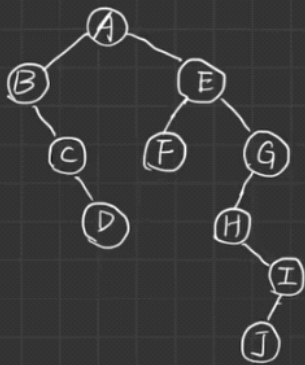
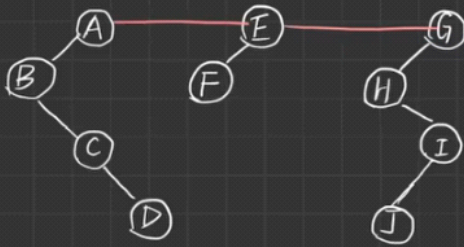
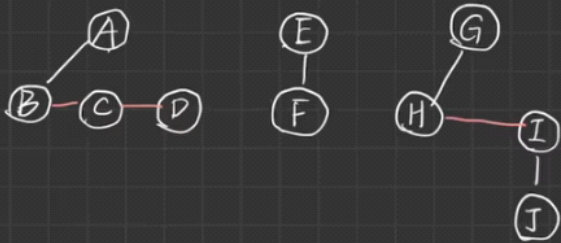
左孩右右连双亲，去掉原来右孩线



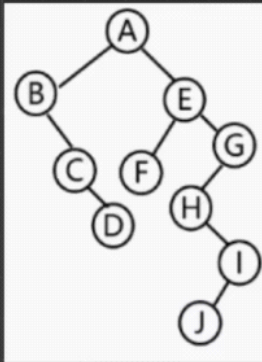
3. 森林 $\rightarrow$ 二叉树 树变二叉根相连



树 $\rightarrow$ 二叉树 兄弟相连留长子, 转45°

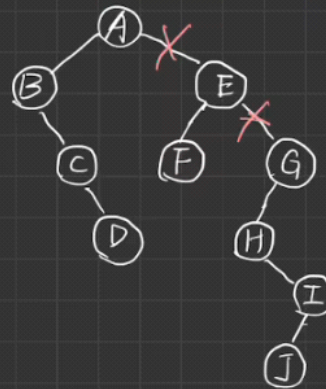


4. 二叉树 $\Rightarrow$ 森林



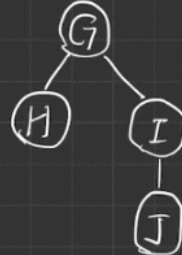
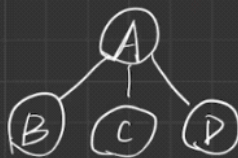
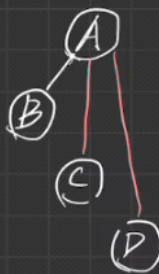
去掉全部右弦线，孤立二叉再还原

根结点与其右孩子，及沿右分支搜索到的所有右孩子间的连线



二叉树 $\Rightarrow$ 树

左孩右右连双亲，去掉原来右弦线



并查集

2025年1月3日 20:11

Union 操作的优化

```
#define SIZE 13
int UFsets[SIZE]; //集合元素数组

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }
}

//Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0) //循环寻找x的根
        x=S[x];
    return x; //根的S[]小于0
}
```

Union操作优化后，
Find 操作最坏时间
复杂度： $O(\log_2 n)$

该方法构造的树高不超过 $\lceil \log_2 n \rceil + 1$

```
//Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    //要求Root1与Root2是不同的集合
    if(Root1==Root2) return;
    //将根Root2连接到另一根Root1下面
    S[Root2]=Root1;
}
```

优化

```
//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]) { //Root2结点数更少
        S[Root1] += S[Root2]; //累加结点总数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点总数
        S[Root1]=Root2; //小树合并到大树
    }
}
```

王道考研/CSKAOYAN.COM

并查集的优化

```
#define SIZE 13
int UFsets[SIZE]; //集合元素数组

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }
}
```

```
//Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0) //循环寻找x的根
        x=S[x];
    return x; //根的S[]小于0
}
```

```
//Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    //要求Root1与Root2是不同的集合
    if(Root1==Root2) return;
    //将根Root2连接到另一根Root1下面
    S[Root2]=Root1;
}
```

核心思想：尽
可能让树变矮

Find优化
(压缩路径)

Union优化

```
//Find “查”操作优化，先找到根节点 再进行“压缩路径”
int Find(int S[],int x){
    int root = x;
    while(S[root]>=0) root=S[root]; //循环找到根
    while(x!=root){ //压缩路径
        int t=S[x]; //t指向x的父节点
        S[x]=root; //x直接挂到根节点下
        x=t;
    }
    return root; //返回根节点编号
}
```

```
//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]) { //Root2结点数更少
        S[Root1] += S[Root2]; //累加结点总数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点总数
        S[Root1]=Root2; //小树合并到大树
    }
}
```


并查集的优化

用数组、双亲表示法来描述元素之间的集合关系。根节点为 -1，非根节点指向父节点下标。

Find (int S[], x) —— 找到 x 所属集合
Union(int S[], int x, int y) —— 将 x、y 所属集合合并

Union
优化

用根节点的负值表示一棵树的结点总数

每次 Union 操作让小树合并到大树根节点下面

Find
优化

“压缩路径”的策略，每次 Find 操作先找到 x 所属根节点，再将查找路径上的所有结点都直接挂在根节点下面

最坏时间复杂度：

Find 操作 = 最坏树高 = $O(n)$

将 n 个独立元素通过多次 Union 合并为一个集合 —— $O(n^2)$

最坏时间复杂度：

Find 操作 = 最坏树高 = $O(\log_2 n)$

将 n 个独立元素通过多次 Union 合并为一个集合 —— $O(n \log_2 n)$

最坏时间复杂度：

Find 操作 = 最坏树高 = $O(\alpha(n))$

将 n 个独立元素通过多次 Union 合并为一个集合 —— $O(n \alpha(n))$

排序

2024年11月20日 18:04

插入排序：

时间复杂度：Insert： $O(i)$
InsertSort： $O(n^2)$

变化：折半插入

链式插入

```
template<class T>
void Insert(const T& e, T* a, int i)
{
    a[0] = e; // 边界, 让i不会<=0
    while (e < a[i])
    {
        a[i + 1] = a[i];
        i--; // 从右往左插入
    }
    a[i + 1] = e; // insert e
}

template<class T>
void InsertSort(T* a, const int n)
{
    for (int j = 2; j <= n; j++)
    {
        T temp = a[j];
        Insert(temp, a, j - 1);
    }
}
```

快速排序：

- 首先找一个轴枢，在它左边的序列都小于它，右边的子序列均大于它，递归子序列。

- 平均性能最好： $O(n \log n)$ 最坏的情况： $O(n^2)$

- 选择子表第一个元素(a[left])作为轴枢：
(变化：以第一个，中间的，最后一个的中间值为轴枢，见cpp)

```
void QuickSort(T* a, const int left, const int right)
{
    if (left < right)
    {
        int i = left, j = right + 1, pivot = a[left];
        do
        {
            do i++;
            while (a[i] < pivot); // 从左边开始, 找到第一个大于等于枢轴的元素
            do j--;
            while (a[j] > pivot); // 从右边开始, 找到第一个小于等于枢轴的元素
            if (i < j)
                swap(a[i], a[j]); // 交换这两个元素, 若i>j, 不交换。
        } while (i < j);
        swap(a[left], a[j]); // 将枢轴与中间元素交换

        QuickSort(a, left, j - 1);
        QuickSort(a, j + 1, right);
    }
}
```

归并排序：

将两个有序的子序列归并为一个有序的子序列。迭代/递归

- 迭代归并： $O(n \log n)$
//把有序表[l:m]和[m+1:n]归并为[l:n]
template<class T>

```

void Merge(T* initList, T* mergedList, const int l, const int m, const int n)
{
    int i1 = l, iResult = l, i2 = m + 1;
    for (; i1 <= m && i2 <= n; iResult++)
    {
        if (initList[i1] <= initList[i2])
        {
            mergedList[iResult] = initList[i1];
            i1++;
        }
        else
        {
            mergedList[iResult] = initList[i2];
            i2++;
        }
    }
    //把最后多出来的部分与前面的合并
    copy(initList + i1, initList + m + 1, mergedList + iResult);
    copy(initList + i2, initList + n + 1, mergedList + iResult);
}

// n:数组的总长度, s:当前子数组的大小(步长)
template<class T>
void MergePass(T* initList, T* resultList, const int n, const int s)
{
    int i = 1;
    // n - 2 * s + 1 确保在最后一轮合并时不会超出数组的边界。
    //在每次合并操作后, i 移动的步长是 2 * s, 因为每次合并都是将两个长度为 s 的子数组合并为一个长度为 2 * s 的有序子数组。
    for (; i < n - 2 * s + 1; i += 2 * s)
    {
        Merge(initList, resultList, i, i + s - 1, i + 2 * s - 1);
    }

    //数组中最后的几个元素可能无法形成完整的两个子数组。
    if ((i + s - 1) < n)
        Merge(initList, resultList, i, i + s - 1, n); //将[i:i+s-1]和最后的[:n]合并
    else
        // 如果 i + s - 1 超出数组范围, 表示当前剩余的子数组无法与下一个子数组完整地进行合并。
        // 直接将剩余部分复制到 resultList。
        copy(initList + i, initList + n + 1, resultList + i);
}

//迭代归并
template<class T>
void MergeSort(T* a, const int n)
{
    T* temp = new T[n + 1];
    for (int l = 1; l < n; l *= 2)
    {
        MergePass(a, temp, n, l); //将长度为 l 的子数组进行合并, 将结果存储在 temp 中
        l *= 2;
        MergePass(temp, a, n, l); //将长度为 2 * l 的子数组进行合并, 将结果存储在 a 中
    }
    delete[] temp;
}

```

堆排序:

时间复杂度: $O(n\log n)$

- **初始堆构建:** 在排过一次完整的堆构建过程, 将数组调整为一个大顶堆。这个过程从最后一个非叶子节点开始, 向上逐步调整, 使得每个子树都满足堆的性质。
- **排序过程中重新调整堆:** 在每次将堆顶元素(最大值)移到数组的末尾后, 通过 `Adjust` 函数重新调整剩余的堆结构, 同时减少堆的大小, 保持堆的性质。具体来说, 当堆顶元素与末尾元素交换后, 新的堆顶元素可能不再满足堆的性质, 此时需要通过调用 `Adjust` 函数重新调整堆顶及其子节点, 确保堆结构的正确性。

```

template<class T>
void Adjust(T* a, const int root, const int n)
{
    T e = a[root];
    int j;
    for (j = 2 * root; j <= n; j *= 2)
    {
        if (j < n && a[j] < a[j + 1])

```

```

        j++;
        if (e >= a[j])
            break;
        a[j / 2] = a[j];
    }
    a[j/2] = e;
}

template<class T>
void HeapSort(T* a, const int n)
{
    // 构建大顶堆
    for (int i = n / 2; i >= 1; i--)
        Adjust(a, i, n);

    // 排序
    for (int i = n - 1; i >= 1; i--)
    {
        swap(a[1], a[i + 1]);
        Adjust(a, 1, i);
    }
}

```

排序类别	排序算法	时间复杂度			辅助空间 复杂度	稳定性
		最好	最坏	平均		
插入排序	直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	折半插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	希尔排序	$O(n)$	$O(n^2)$	$O(n^{1.3})$	$O(1)$	不稳定
交换排序	冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
	快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$	不稳定
选择排序	简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
归并排序		$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定

基数排序

2024年12月17日 16:50

其他排序方式（书上没有的）

2024年12月23日 14:21

了解外部排序的过程

2024年12月23日 14:22

希尔排序

$d = 5$

7	27	20	60	44	16	28	36	67	55	36
---	----	----	----	----	----	----	----	----	----	----

$d = 2$

7	16	20	27	28	36	36	55	44	60	67
---	----	----	----	----	----	----	----	----	----	----

$d = 1$

7	16	20	27	28	36	36	44	55	60	67
0	1	2	3	4	5	6	7	8	9	10

增量序列
 $(\frac{n}{2}, \frac{n}{4}, \frac{n}{8} \dots 1)$

缩小增量排序

不稳定

最好： $O(n)$

最坏： $O(n^2)$ 目前尚无定论

平均： $O(n^{1.3})$

辅助空间： $O(1)$

贪婪算法—最小代价生成树

2024年11月24日 13:17

逐步构建代价最小的树

克鲁斯卡尔算法：

小顶堆排序

选择代价最小的边，构建森林

判断有无环：已构建的边的顶点加入集合，若新加的边的两个顶点均在集合里，舍弃

```
1 T= {}
2 while (T contains less than n-1 edges) && (E not empty)) {
3   choose an edge (v, w) from E of lowest cost;
4   delete (v, w) from E;
5   if ((v, w) does not create a cycle in T) add (v, w) to T;
6   else discard (v, w);
7 }
8 if (T contains fewer than n-1 edges) cout << "no spanning tree" << endl;
```

构建小顶堆: O (e)

小顶堆确定并删除一条边: O(log_e)

检查环：

- 在每次选择边时，使用并查集（Union-Find）数据结构来检查是否会形成环。具体步骤是：
 - 查找这条边的两个端点所在的集合（即根节点）。
 - 如果这两个端点在不同的集合中，则说明添加这条边不会形成环，将这两个集合合并。
 - 如果这两个端点在同一个集合中，则添加这条边会形成环，因此跳过这条边。

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// 边结构体，包含起点、终点和权重
struct Edge {
    int src, dest, weight;
};

// 并查集结构体，包含父节点和秩
struct DisjointSets {
    int* parent, * rank;
    int n;

    // 构造函数，初始化并查集
    DisjointSets(int n) {
        this->n = n;
        parent = new int[n + 1];
        rank = new int[n + 1];

        // 初始化每个节点为自己的父节点，秩为0
        for (int i = 0; i <= n; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }

    // 查找函数，带路径压缩
    int find(int u) {
        if (u != parent[u])
            parent[u] = find(parent[u]);
        return parent[u];
    }

    // 合并两个集合，带秩优化
    void merge(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX != rootY) {
            if (rank[rootX] > rank[rootY])
                parent[rootY] = rootX;
            else if (rank[rootX] < rank[rootY])
                parent[rootX] = rootY;
            else {
                parent[rootY] = rootX;
                rank[rootX]++;
            }
        }
    }
};

// 克鲁斯卡尔算法实现
void KruskalMST(vector<Edge>& edges, int V) {
    vector<Edge> result; // 保存结果的边
    int e = 0; // 初始化结果中的边数
    int i = 0; // 初始化已处理的边数

    // 按权重排序所有边
    sort(edges.begin(), edges.end(), [](Edge a, Edge b) {
        return a.weight < b.weight;
    });

    // 创建并查集
    DisjointSets ds(V);

    // 遍历所有边，按照权重从小到大
    while (e < V - 1 && i < edges.size()) {
        // 选取权重最小的边
        Edge next_edge = edges[i++];

        int x = ds.find(next_edge.src);
        int y = ds.find(next_edge.dest);

        // 如果选取这条边不形成环，将其加入结果
        if (x != y) {
            result.push_back(next_edge);
            ds.merge(x, y);
            e++;
        }
    }

    // 输出结果
    cout << "构成最小生成树的边及其权重:\n";
    for (i = 0; i < result.size(); ++i)
        cout << result[i].src << " -- " << result[i].dest << " == " << result[i].weight
        << endl;
}

int main() {
    int V = 5; // 图中的节点数
    vector<Edge> edges = { {0, 1, 10}, {0, 2, 6}, {0, 3, 5}, {1, 3, 15}, {2, 3, 4} };

    KruskalMST(edges, V);

    return 0;
}
```

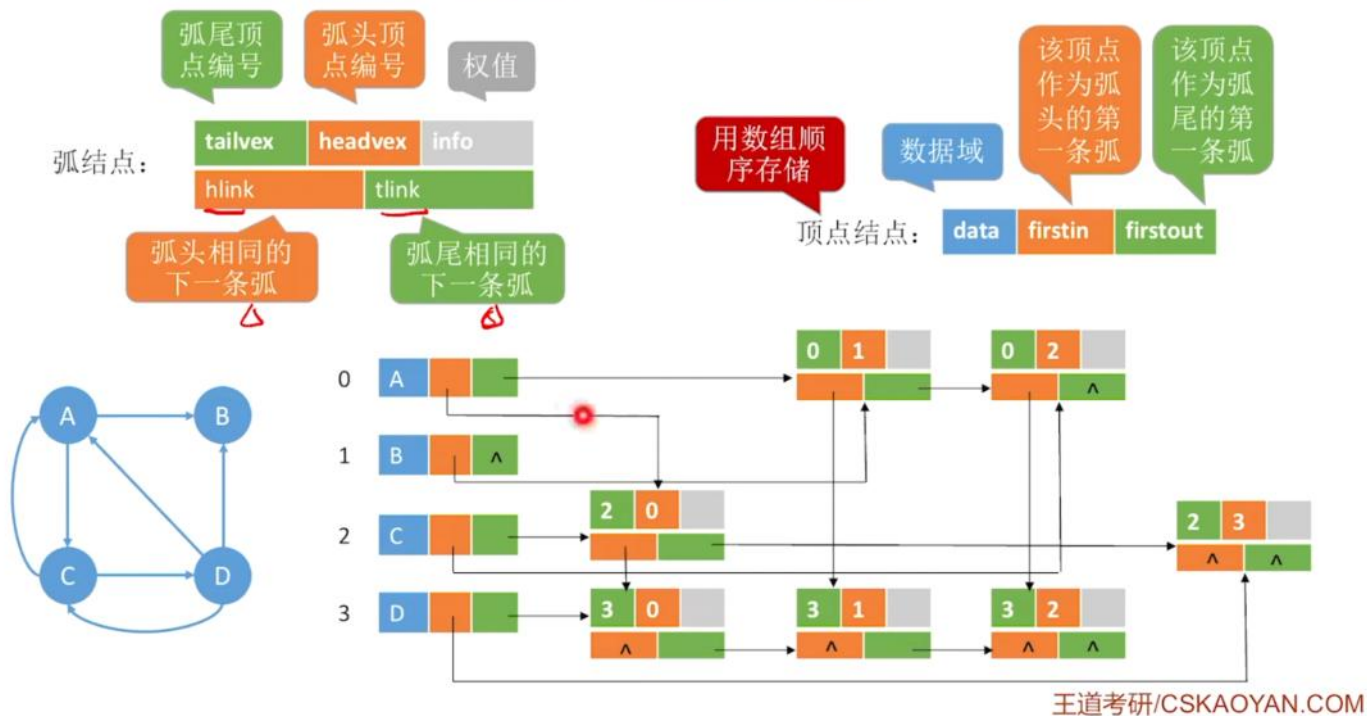
普里姆算法:

逐条边的构造, 始终是一棵树

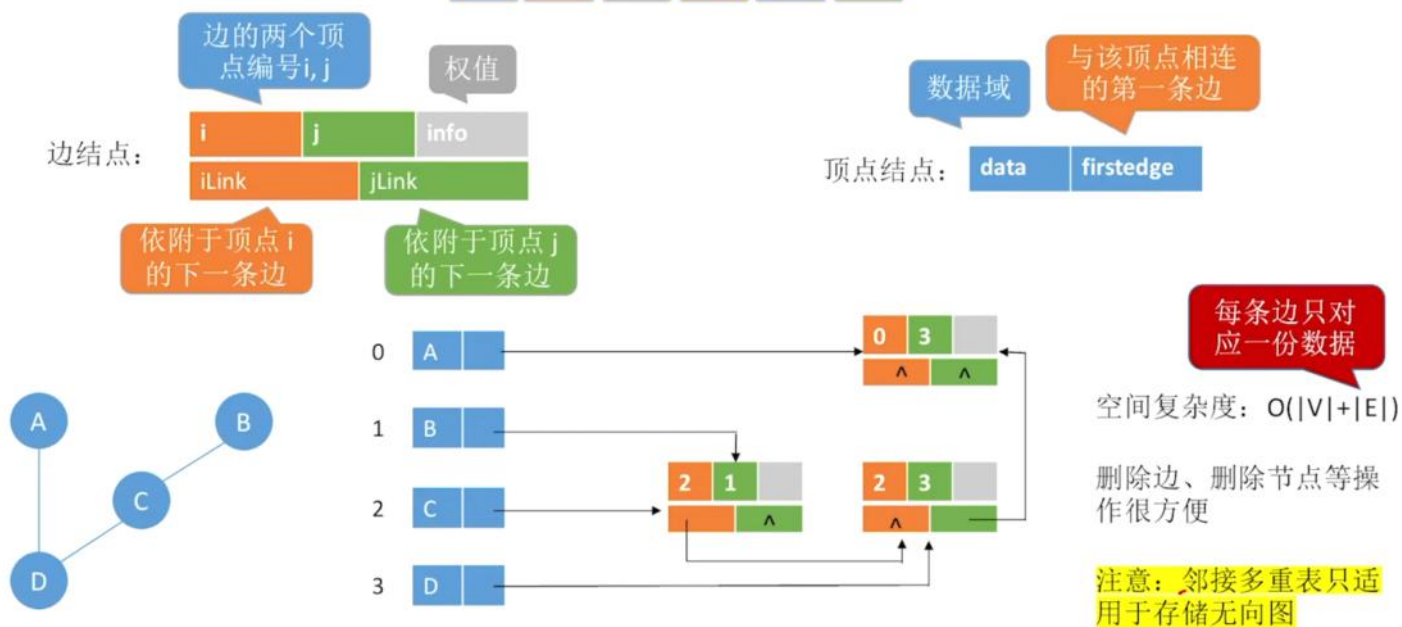
```
// Assume that G has at least one vertex.
TV={0};      // start with vertex 0 and no edges
for (T= $\emptyset$ ; T contains fewer than n-1 edges; add (u, v) to T)
{
    Let (u, v) be a least-cost edge such that  $u \in TV$  and  $v \notin TV$ ;
    if (there is no such edge) break;
    add v to TV;
}
if (T contains fewer than n-1 edges) cout<<"no spanning tree"<<endl;
```

$O(n^2)$

十字链表存储有向图



邻接多重表存储无向图



Dijkstra算法—最短路径

2025年1月4日 22:22

从一个点到其他点的最短路径

哈希函数

2024年12月19日 23:04

1.除留取余法

2.平方取中法

3.折叠法

4.数字分析法

5.将关键字转化为整数

溢出处理

2024年12月20日 17:55

1. 开放定址法

线性探测法

二次探测法

再散列法

随机探测法

2. 链地址法

AVL树

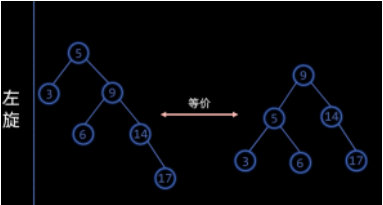
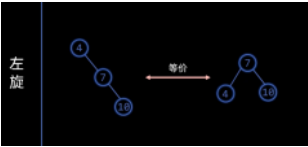
2024年12月16日 22:36

搜索，插入，构建

平衡因子：左子树高度-右子树高度

AVL树：所有节点的平衡因子为 (-1, 0, 1)

左旋：冲突的左孩子→右孩子
右旋：冲突的右孩子→左孩子

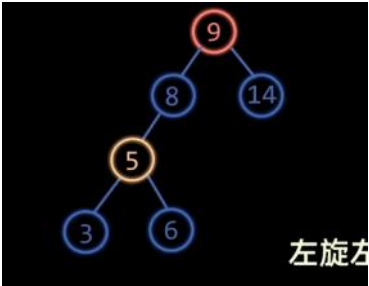
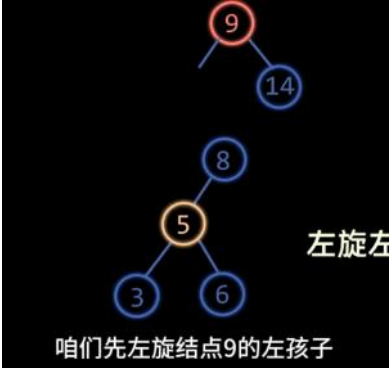
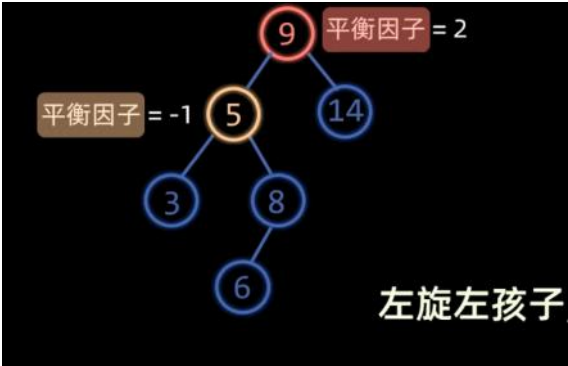
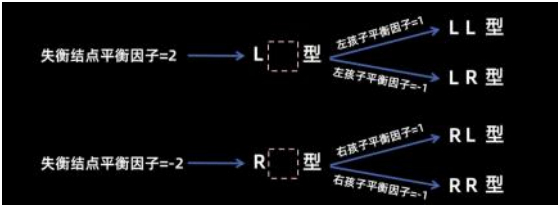


```
对node左旋: if(node->right->left)
{
    AVL* temp=node->right->left;
    node->right->left=node;
    node->right=temp;
}

把节点挪到右子节点的左子节点上
如果右子节点本来就存在左子节点 //不需要判断
    把它移到节点的右子节点上
将右子节点和节点的父节点连接(如果节点的父节点存在)
```

- LL型：特点：失衡节点的平衡因子为2，失衡节点的左孩子的平衡因子为1
操作：将失衡节点右旋
- RR型：特点：失衡节点的平衡因子为-2，失衡节点的右孩子的平衡因子为-1
操作：将失衡节点左旋
- LR型：特点：失衡节点的平衡因子为2，失衡节点的左孩子的平衡因子为-1
操作：先将失衡节点的左孩子左旋，再将失衡节点右旋
- RL型：特点：失衡节点的平衡因子为-2，失衡节点的右孩子的平衡因子为1
操作：先右旋右孩子，再左旋失衡节点

四种失衡情况			
LL型	RR型	LR型	RL型
失衡结点: 平衡因子 = 2 失衡结点左孩子: 平衡因子 = 1 右旋	失衡结点: 平衡因子 = -2 失衡结点右孩子: 平衡因子 = -1 左旋	失衡结点: 平衡因子 = 2 失衡结点左孩子: 平衡因子 = -1 左旋左孩子, 然后右旋	失衡结点: 平衡因子 = -2 失衡结点右孩子: 平衡因子 = 1 右旋右孩子, 然后左旋



插入过程中，多个祖先节点失衡：只需调整离插入节点最近的祖先节点



删除：O(logn)

// 删除AVL树中的节点

```
TreeNode* remove(TreeNode* node, int k)
{
    // 如果节点为空，直接返回
    if (!node)
        return node;

    // 递归寻找要删除的节点
    if (k < node->value)
        node->left = remove(node->left, k);
    else if (k > node->value)
        node->right = remove(node->right, k);
    else
    {
        // 找到要删除的节点

        // 如果节点有一个子节点或没有子节点
        if (!node->left || !node->right)
        {
            TreeNode* temp = node->left ? node->left : node->right;

            // 如果没有子节点
            if (!temp)
            {
                temp = node;
                node = nullptr;
            }
            // 如果只有一个子节点
            else
                *node = *temp;

            delete temp;
        }
        else
        {
            // 如果节点有两个子节点
            TreeNode* temp = getMin(node->right); // 找到右子树中的最小值节点
            node->value = temp->value; // 用最小值替换当前节点的值
            node->right = remove(node->right, temp->value); // 删除右子树中的最小值节点
        }
    }

    // 如果树只有一个节点
    if (!node)
        return node;

    // 更新当前节点的高度
    node->height = 1 + max(getHeight(node->left), getHeight(node->right));
    int balance = getBalance(node); // 获取当前节点的平衡因子

    // LL
    if (balance > 1 && getBalance(node->left) >= 0)
        return rotateRight(node);

    // LR
    if (balance > 1 && getBalance(node->left) < 0)
    {
        node->left = rotateLeft(node->left);
        return rotateRight(node);
    }

    // RR
    if (balance < -1 && getBalance(node->right) <= 0)
        return rotateLeft(node);

    // RL
    if (balance < -1 && getBalance(node->right) > 0)
    {
        node->right = rotateRight(node->right);
        return rotateLeft(node);
    }

    // 返回当前节点
    return node;
}
```

在 AVL 树中删除节点并保持平衡的过程，关键在于通过递归的方法逐层返回调整后的节点给其父节点。

在代码中，每次删除节点或进行旋转操作后，都会返回调整后的子树根节点。这个根节点在上一级调用中被赋值给相应的子树位置，从而实现逐层返回调整后的节点。

在AVL树的节点删除过程中，递归是实现逐层向上检查和平衡的关键。这个过程如下：

1. **递归删除节点**：从树的根节点开始，根据删除的键值k，递归地找到目标节点。当目标节点被删除或替换后，函数会返回调整后的子树根节点。
2. **逐层返回**：删除节点之后，函数并不会立即结束，而是继续向调用它的父节点返回。每次返回时，都会更新当前节点的高度，并检查树的平衡情况。
3. **更新高度**：每次递归返回时，先更新当前节点的高度。

```
cpp
node->height = 1 + max(getHeight(node->left),
    getHeight(node->right));
```
4. **检查平衡**：计算当前节点的平衡因子，并根据平衡因子的值决定是否需要旋转调整树的平衡。

```
cpp
int balance = getBalance(node);

if (balance > 1 && getBalance(node->left) >= 0)
    return rotateRight(node);
if (balance > 1 && getBalance(node->left) < 0) {
    node->left = rotateLeft(node->left);
    return rotateRight(node);
}
if (balance < -1 && getBalance(node->right) <= 0)
    return rotateLeft(node);
if (balance < -1 && getBalance(node->right) > 0) {
    node->right = rotateRight(node->right);
    return rotateLeft(node);
}
```
5. **返回调整后的节点**：每次检查并调整当前节点后，将调整后的节点返回给其父节点。

```
cpp
return node;
```

这个过程会在递归返回时逐层更新和调整，从叶子节点一直到根节点。因此，AVL树能够保证在 $O(\log n)$ 时间复杂度内保持平衡。

删除操作的过程

删除一个节点 k 的过程可以分为以下几步：

1. 找到要删除的节点 k ：

在二叉搜索树中查找一个节点的时间复杂度是 $O(\log n)$ ，因为树的高度是 $O(\log n)$ 。

2. 删除节点 k ：

- 如果 k 是叶子节点，直接删除即可。
- 如果 k 有一个子节点，用子节点替换 k 。
- 如果 k 有两个子节点，用 k 的前驱（左子树中的最大节点）或后继（右子树中的最小节点）替换 k ，然后递归地删除前驱或后继。查找前驱或后继的过程也是 $O(\log n)$ 的。

3. 重新平衡树

删除操作可能会导致 AVL 树失去平衡。因此，需要执行旋转操作来重新平衡树。每个旋转操作的时间复杂度是 $O(1)$ ，但是删除一个节点后，最多需要 $O(\log n)$ 次旋转，因为树的高度是 $O(\log n)$ 。

总时间复杂度

综合上述步骤，AVL 树中删除节点 k 的总时间复杂度如下：

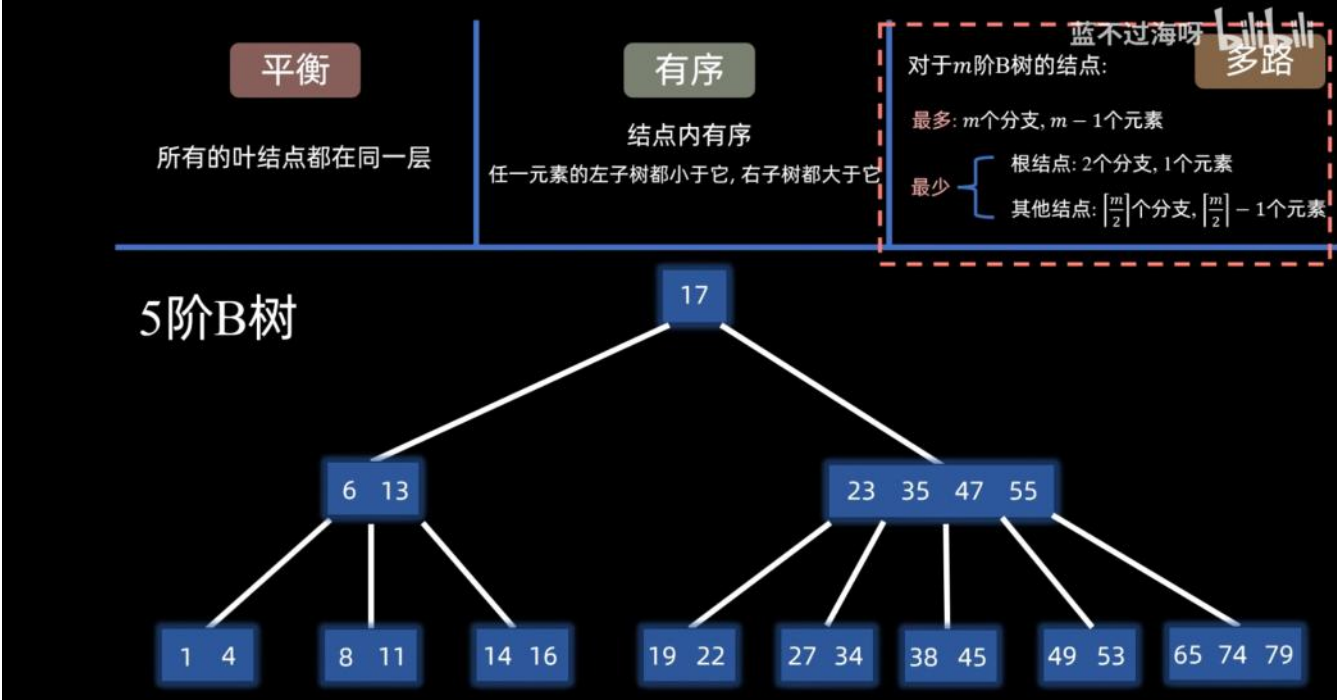
- 查找节点 k ： $O(\log n)$
- 删除节点 k ： $O(\log n)$
- 重新平衡树： $O(\log n)$

因此，删除操作的总时间复杂度是： $O(\log n) + O(\log n) + O(\log n) = O(\log n)$

综上所述，AVL 树中的删除操作可以在 $O(\log n)$ 的时间内完成，其中 n 是树的节点数目。

多路搜索树

2024年12月23日 14:27



删除非叶结点元素最终都转换成删除叶结点元素



B+树

2024年12月23日 14:29

不是重点

KMP算法

2024年12月26日 22:37



字符串s
模式串p
线性时间复杂度

定义：设 $p=p_0 p_1 \dots p_{n-1}$ 是一个模式串，定义它的失配函数 (failure function), f 为：
$$f(j)=\begin{cases} k & \text{为满足 } p_0 \dots p_k = p_{j-k} \dots p_j \text{ 且 } k < j \text{ 的最大整数, 这样的 } k \text{ 存在且 } k \geq 0 \\ -1 & \text{上述 } k \text{ 不存在} \end{cases}$$

S

A	B	C	A	C	B	A	B	B	A	C	A	B	C	A	C	
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	--

找p的最长的公共前后缀

0	1	2	3	4	5	6	7	8	9
a	b	c	a	b	c	a	c	a	b
-1	-1	-1	0	1	2	3	-1	0	1

-1	-1	-1	0	1	2	3	-1	0	1
----	----	----	---	---	---	---	----	---	---

根据失配函数的定义，可以得出如下的模式匹配规则：当出现形如 $s_{i-j} \dots s_{i-1} = p_0 \dots p_{j-1}$ 且 $s_i \neq p_j$ 的部分匹配时，若 $j \neq 0$ ，则下一轮模式匹配从比较 s_i 和 $p_{f(j-1)+1}$ 开始；若 $j = 0$ ，则继续比较 s_{i+1} 和 p_0 。根据这种模式匹配规则可写出 FastFind 函数（见程序2.16）。FastFind 函数利用整型数组 f 表示失配函数。该数组是 String 的一个私有数据成员。

对 FailureFunction 函数的分析：当 while 循环每次重复时， i 的值都会减少（根据失配函数 f 的定义）。每当 for 循环的一次重复开始时，变量 i 将被复位。它或者被复位成 -1 （当 $j=1$ 或者前一次循环时执行了第 11 行），或者被复位成比前一次循环结束时的终值大 1 的值（即前一次循环执行到了第 10 行）。由于第 7 行仅执行 $\text{lengthP}-1$ 次，所以 i 的总增量最多为 $\text{lengthP}-1$ ，它减少的次数也不能多于 $\text{lengthP}-1$ 次。因此，while 循环所重复的次数在整个算法中最多为 $\text{lengthP}-1$ 次，于是 FailureFunction 函数的计算时间为 $O(\text{lengthP})$ 。□

注意：当失配函数没有被事先计算出来时，模式匹配过程将包括首先利用 FailureFunction 计算失配函数，然后使用 FastFind 进行模式匹配。总的执行时间将是 $O(\text{lengthP}+\text{lengthS})$ 。

2024考试重点

2024年12月23日 14:02

1.3
1.5
1.7

2.2
2.3
2.4
2.6 KMP算法

概念+应用

3.2
3.3
3.5
3.6

10*2选择

6*8简答

3 算法设计

4.4
4.2
4.4
4.5
4.6
4.7
4.9
4.10

(思路, 性能)

线性结构+树+图

5